NLP Clinical Note Summarization: Full Project Document

Fine-tuning BART-large-cnn for structured clinical summaries. Reconstructed from the recovered notebook (finalnlp2.ipynb) and the IEEE paper (NLP_Final.pdf).

1. Project Overview

Title: NLP Based Patient Record Summarization for Clinical Decision

Authors: Aniket Patil, Sujal Revankar, Viraj Patil, Srushti Bandi and Savita Bagewadi (Department of Computer Science and Engineering (AI), KLE Technological University, Dr. M. S. Sheshgiri Campus, Belagavi)

Goal: Convert long unstructured clinical notes into structured JSON-style summaries covering visit motivation, admission details, patient information, medical history, symptoms, examinations, diagnosis and treatment.

Approach: Three systems were compared on the same held-out notes: an extractive TF-IDF baseline, zero-shot BART-large-cnn and a fine-tuned BART-large-cnn.

Headline result: Fine-tuned BART reached ROUGE-1 0.4922 and BERTScore-F1 0.9187 and beat both baselines on every overlap metric. It scored lower than both baselines on the custom factual consistency proxy (0.58 against about 1.0).

Environment: Kaggle notebook on an NVIDIA Tesla T4 (two GPUs reported). Executed on 2026-04-20.

2. Dataset

Item	Detail
Name	AGBonnet/augmented-clinical-notes (Hugging Face)
Size	30,000 records
Columns	idx, note, full_note, conversation, summary
Input used	note

Item	Detail
Target used	summary (JSON-style structured clinical summary)
Not used	full_note and conversation
Sample drawn	5,000 records (random seed 42)
Split	3,500 train / 750 validation / 750 test (70/15/15)
Null check	0 missing values in note and summary

Provenance: The full_note field originates from PMC-Patients (case reports published in PubMed Central). The conversation field comes from NoteChat (synthetic patient-doctor dialogue). Neither of those two fields feeds the model.

Measured text statistics (notebook EDA):

Statistic	Notebook value	Paper Table I value
Median note length	316 words	about 220 words
Median summary length	446 words	about 180 words
Median compression ratio	1.407 (summary longer than note)	not reported

The summaries are longer than the notes because the JSON schema carries many fields filled with "None" or "Not mentioned". This is why the target length was raised to 384 tokens.

3. Preprocessing

1. Keep only the note and summary columns and drop rows with missing values.
2. Normalize whitespace with a regular expression and strip leading and trailing spaces.
3. Remove examples where either text is 10 characters or shorter.
4. Sample 5,000 rows with seed 42 and split with scikit-learn train_test_split (30% held out then divided 50/50).
5. Tokenize with the facebook/bart-large-cnn tokenizer: maximum 512 input tokens and maximum 384 target tokens with padding to max length.
6. Replace padding token ids in labels with -100 so the loss ignores them.

7. Wrap the splits in a Hugging Face DatasetDict.

4. Models Compared

4.1 TF-IDF extractive baseline

- Split the note into sentences and keep sentences longer than 4 words.
- Build a TF-IDF matrix with English stop words removed.
- Score each sentence by cosine similarity to the mean document vector.
- Return the top 5 sentences in their original order.
- Strength: perfectly grounded in the source. Weakness: no abstraction and no structure.

4.2 Zero-shot BART-large-cnn

- Pretrained facebook/bart-large-cnn (trained on CNN/DailyMail news) with no fine-tuning.
- Input truncated to the first 1,024 characters. Generation with max_length 256 and min_length 60 and no sampling.
- Output is fluent prose and does not follow the JSON schema.

4.3 Fine-tuned BART-large-cnn (proposed)

- Same checkpoint (406,290,432 parameters) fine-tuned as a sequence-to-sequence model on note to structured summary pairs.
- Inference: beam search with 4 beams and length penalty 1.2 and no_repeat_ngram_size 3 and max_length 384 and min_length 80.
- Saved to ./bart_large_medical_summarizer_final.

5. Training Configuration

Parameter	Value
Base model	facebook/bart-large-cnn
Learning rate	3e-5

Parameter	Value
Scheduler	Cosine with 131 warmup steps (6% of a computed 2,185 total)
Per-device batch size	2 (train and eval)
Gradient accumulation	4
Epochs	up to 5 with early stopping (patience 2)
Weight decay	0.01
Label smoothing	0.1
Precision	fp16
Evaluation and checkpointing	every epoch
Best model selection	lowest eval_loss (load_best_model_at_end)
Checkpoints kept	2
Seed	42
Libraries	transformers 4.35.2 and accelerate 0.24.1 and PyTorch

Runtime: 5,777.6 seconds (about 96 minutes). **Final reported train loss:** 1.6406.

Training log captured from the notebook

Epoch (as logged)	Training loss	Validation loss
0	1.6846	1.6575
1	1.6138	1.6208
2	1.5646	1.6035
4	1.5223	1.6013

The loss curve figure shows training loss falling from about 2.7 to about 1.52 and validation loss falling from about 1.66 to about 1.60 with a flat tail. The gap between the two curves is small.

Inference from step arithmetic (not stated in the notebook)

The script computed total steps as 3,500 / (2 x 4) x 5 = 2,185 as if one GPU were used. The notebook reports two T4 GPUs and the loss plot ends near step 1,090 with validation points about 220 steps apart. That pattern fits an effective batch of 16 (2 GPUs x 2 x 4) with about 219 steps per epoch and 5 completed epochs. If correct then:

- The effective batch size was probably 16 and not 8 as the paper states.
- The cosine schedule was planned for 2,185 steps but ran only about half of them so the learning rate never decayed fully.
- Early stopping most likely did not trigger.

Treat this as a likely reading to verify against the Kaggle run and not as a confirmed fact.

6. Evaluation Methodology

All metrics were computed against the reference JSON summaries on the **first 50 test notes** (N_EVAL = 50).

Metric	How it was actually implemented
ROUGE-1/2/L	rouge-score library with stemming and F-measure averaged per sample
BERTScore-F1	bert-score with distilbert-base-uncased
METEOR	NLTK meteor_score on lowercased word tokens
NER preservation	Intended: scispaCy en_core_sci_sm. Actual: the model failed to load so a 12-pattern regex keyword fallback was used. Score is the share of reference keywords found in the prediction text.
MEDCON	Custom lexicon of about 150 terms in six groups (diseases and conditions and symptoms and medications and procedures and diagnostics and anatomy) with substring matching. F1 of shared concepts. Not the official UMLS-based MEDCON.
Factual consistency	Fraction of content tokens in the prediction that also occur in the source note (stop words removed). A grounding proxy and not an NLI-based factuality score.

7. Results

7.1 Main table (50 test notes)

Model	R-1	R-2	R-L	BERT-F1	METEOR	NER	MEDCON	Factual
TF-IDF extractive	0.2918	0.1751	0.1936	0.7291	0.0684	0.2459	0.5143	1.0000
BART zero-shot	0.1926	0.1169	0.1384	0.7128	0.0395	0.1814	0.4268	0.9995
Fine-tuned BART	**0.4922**	**0.3381**	**0.3470**	**0.9187**	**0.2546**	**0.5818**	**0.6531**	0.5802

7.2 Relative gains of fine-tuned BART (paper Table IV)

Metric	Gain vs TF-IDF	Gain vs zero-shot
ROUGE-1	+69%	+156%

Metric	Gain vs TF-IDF	Gain vs zero-shot
ROUGE-2	+93%	+189%
BERTScore-F1	+26%	+29%
METEOR	+272%	+545%
NER	+137%	+221%
MEDCON	+27%	+53%

7.3 Interpretation

- The fine-tuned model wins by a wide margin on all overlap and semantic metrics because it learns the JSON schema and field vocabulary. The baselines produce prose that cannot overlap well with a structured reference.
- The low factual consistency score (0.58) is largely a formatting effect. JSON keys and filler values such as "not mentioned" do not appear in the note and so count as ungrounded tokens. It is still worth reporting honestly because it also reflects real invented content (see 8.1).
- High BERTScore for the fine-tuned model is partly inflated by shared JSON boilerplate between prediction and reference.

8. Qualitative Analysis

8.1 Observed fine-tuned outputs

Test note	What the model did well	Observed problems
Sample 1 (45-year-old male with cirrhosis and flank pain)	Correct visit motivation and age and sex and admission date	Admission reason given as subacute cholecystitis while the reference says septic shock secondary to recurrent cholecystitis. Medical history placed under the wrong field (socio economic context). Malformed quote at family medical history.

Test note	What the model did well	Observed problems
Sample 2 (75-year-old with bladder cancer history)	Correct visit motivation and age and date	Family medical history filled with "numerous" with no support in the note. Admission reason copied from past history.
Sample 3 (32-year-old with maxillary swelling)	Correct visit motivation and demographics	Height filled with "noncontributory" (a phrase from the note placed in the wrong field). Malformed key quoting.
Demo (27-year-old woman with nasal tumor)	Captured symptoms and tests and treatments	Broken JSON syntax throughout (misplaced quotes and missing colons and bracket errors). Family medical history set to "unremarkable" from the patient's own history.

8.2 Failure modes

1. **Invalid JSON.** Outputs often cannot be parsed. The paper calls the output structured but no JSON validity rate was measured.
2. **Field misplacement.** Correct facts land under the wrong key.
3. **Hallucinated or over-filled fields** where the reference says None.
4. **Truncation risk** on long notes because the input is cut at 512 tokens.

9. Tools and Resources Used

9.1 Data and models

Resource	Link	Role
Augmented Clinical Notes	https://huggingface.co/datasets/AGBonnet/augmented-clinical-notes	Training and evaluation data

Resource	Link	Role
PMC-Patients	https://huggingface.co/datasets/zhengyun21/PMC-Patients	Origin of the clinical notes (not loaded directly)
BART-large-cnn	https://huggingface.co/facebook/bart-large-cnn	Fine-tuned model and zero-shot baseline

9.2 Libraries

Library	Link	Use
Transformers 4.35.2	https://huggingface.co/docs/transformers	Model and tokenizer and Seq2SeqTrainer and pipelines
Datasets	https://huggingface.co/docs/datasets	Dataset loading and DatasetDict
Accelerate 0.24.1	https://huggingface.co/docs/accelerate	Training backend
PEFT 0.6.2	https://huggingface.co/docs/peft	Installed but not used (no LoRA in the code)
PyTorch	https://pytorch.org	Training and GPU execution
scikit-learn	https://scikit-learn.org	Splits and TF-IDF and cosine similarity
NLTK	https://www.nltk.org	Tokenization and METEOR
rouge-score	https://github.com/google-research/google-research/tree/master/rouge	ROUGE
bert-score	https://github.com/Tiiiger/bert_score	BERTScore
scispaCy	https://github.com/allenai/scispacy	Planned NER (model failed to load)

Library	Link	Use
pandas and NumPy and Matplotlib and Seaborn	https://pandas.pydata.org	Analysis and plots

Also installed but unused: sentencepiece and bert-extractive-summarizer.

9.3 Background literature cited in the paper

BART (Lewis et al.) and BioBART (Yuan et al.) and Med-PaLM style clinical knowledge (Singhal et al.) and Van Veen et al. on clinical summarization and ACI-Bench (Yim et al.) and SummEval (Fabbri et al.) and SummaC (Laban et al.) and ScispaCy (Neumann et al.). These are background references and not proof that each method was implemented.

10. Pipeline Summary

1. Load the dataset from Hugging Face.
2. Clean and sample and split and tokenize.
3. Run the TF-IDF baseline and zero-shot BART on 50 test notes.
4. Fine-tune BART-large-cnn for up to 5 epochs.
5. Generate structured summaries with beam search.
6. Score all three systems with ROUGE and BERTScore and METEOR and NER and MEDCON and factual consistency.
7. Plot comparisons and inspect qualitative samples.

11. Discrepancies Between Paper and Notebook

#	Issue	Paper says	Notebook shows	Suggested fix
1	Dataset length stats	Median note about 220 and summary about 180 words	316 and 446	Replace with measured values
2	Evaluation size	"Held-out test set"	50 of 750 test notes	State N = 50 or rerun on all 750

#	Issue	Paper says	Notebook shows	Suggested fix
3	Factual consistency	Not reported	0.58 for FT vs about 1.0 for baselines	Report it with an explanation
4	NER metric	ScispaCy entity preservation	Regex keyword fallback	Rename as keyword preservation or install en_core_sci_sm and rerun
5	MEDCON	Clinical concept overlap	Custom lexicon with substring match	Describe as lexicon-based approximation
6	Structured output	Output is structured and clinically usable	Frequent invalid JSON	Add a JSON validity rate and temper the claim
7	Hybrid system	Abstract says TF-IDF extraction plus fine-tuned BART	TF-IDF is a separate baseline and is not fed into BART	Say "compared extractive TF-IDF and zero-shot BART and fine-tuned BART"
8	Effective batch size	8	Probably 16 on two GPUs	Check the Kaggle run then correct
9	PEFT	Not mentioned	Installed but unused	Do not claim parameter-efficient tuning
10	Clinical usability	Summaries can be used directly in clinical settings	No clinician review and a 50-note sample	Soften to "potentially useful pending expert review"
11	Conclusion naming	Mentions a "SCNS framework"	No such name in the notebook	Remove or define it

12. Limitations

- Small evaluation sample (50 notes) with no confidence intervals and a single seed.
- Reference summaries are model-generated (augmented dataset) and not clinician-written.

- Custom metrics are proxies and are not validated against human judgement.
- Input truncation at 512 tokens can drop information from long notes.
- No hallucination audit or clinician review.
- Single dataset and no external validation.

13. What Could Not Be Recovered

Item	Status
Fine-tuned weights (bart_large_medical_summarizer_final/)	Not in the uploads. Check the Kaggle notebook Output tab or version history.
Trainer checkpoints in bart_large_medical_summarizer/	Same location and same status
Full per-sample predictions	Only 3 samples and 1 demo were printed
Separate PNG files	Recovered from notebook outputs (EDA and loss curve and ROUGE chart and all-metrics chart)
LaTeX source of the paper	Not recoverable from the PDF. Rebuild it or contact co-authors.

14. Recommended Next Steps

1. Open the original Kaggle notebook and download the saved model folder before it expires.
2. Rerun evaluation on all 750 test notes and add a JSON validity rate.
3. Install en_core_sci_sm and recompute NER preservation.
4. Fix the discrepancies in Section 11 in the paper text.
5. Consider constrained decoding or a JSON repair step to cut syntax errors.
6. Add a small clinician or peer review of 20 to 30 summaries.
7. Upload the code and results to a GitHub repository so the project cannot be lost again.